

Campus Assistant Database Design v2 (中文)

目的

本文将 **ADR-0004** 到 **ADR-0010** 中已经接受的数据库架构决策整合为一份面向实现的设计参考。

它的目标不是替代 ADR。ADR 仍然是长期有效的决策记录。本文存在的意义，是说明这些已接受的决策如何组合成校园助手的一套**连贯数据库架构**。

这份 v2 设计替代了 **db-schema-v1.md** 所暗示的方向。后者更像一个 SIS（学生信息系统）或校园行政后台风格的建模方案。v2 设计则面向一个 **function-call-first** 的校园助手，它需要：

- 少量高精度的结构化对象域
- 对异构校园信息保持广覆盖的 source-first 支撑
- 面向易变内容的 freshness-aware 检索行为
- 一条原生基于 Supabase 的混合检索路径，并最终替代 QMD 成为生产环境的主检索后端

设计目标

数据库应优先优化以下能力：

1. 面向 tool 的高价值领域精确索引

- Courses
- Housing
- Location / service destinations
- Academic calendar items

2. 跨学校、广覆盖、可迁移的 source ingest 能力

- 网页
- feeds
- APIs
- 地图衍生内容
- 手工导入内容
- 导入的历史语料

3. freshness-aware 的答案路由能力

- 稳定信息默认可以使用本地结构化/索引存储
- 易变信息不能把数据库中的归档记录当作默认当前真相

4. 可解释的 provenance（来源链路）

- object-first 记录必须可以追溯回被抓取的 source 与 artifact
- retrieval 命中结果必须可以追溯回权威的 source-derived artifact

5. 跨学校可移植性

- 避免把 schema 做成某个学校特定的 registrar、宿舍分配系统、GIS 平台或行政 ERP 后端

核心架构

已接受的架构包含三层行为层：

1. 少量稳定领域使用 **object-first**
2. 大多数校园信息使用 **source-first**
3. 针对易变领域使用 **freshness-aware routing**

这三层建立在严格的 A1 source-first base model 之上：

```
sources -> source_snapshots -> artifacts -> chunks
```

并由一条 **Supabase-native hybrid retrieval** 路径提供服务，这条路径应成为长期生产检索后端。

第 1 层：Object-First Domains

只有少数几个领域应该获得一等公民式的对象建模。

已批准的 object-first domains

- `course`
- `course_offering`
- `housing`
- `location_or_service`
- `academic_calendar_item`

这些领域之所以采用 object-first，是因为用户需要对它们获得**完整、可过滤、高置信度**的索引。

Object-first 设计规则

- 每个已批准领域都应保持**范围收敛**
- 除非后续 ADR 明确批准，否则不要把 optional facets 提升为独立的一等对象
- object 记录必须通过以下字段保持**source-grounded**：
 - `source_id`
 - `source_snapshot_id`
 - `primary_artifact_id`
- object-first 领域不会替代 source-first 证据层；它们是在其之上提供一个稳定、面向 assistant 的索引

第 2 层：Source-First Domains

大多数校园信息仍然保持 source-first。

例如：

- `guides`

- FAQs
- department pages
- faculty pages
- student-life content
- organization content
- procedural pages
- 一般性叙述内容

这些领域默认不需要 first-class object model。它们应通过 A1 source pipeline 进入系统，并通过 artifact/chunk 索引变得可搜索。

第 3 层：Freshness-Aware Routing

系统不仅要区分“结构化程度”，还必须区分“freshness 行为”。

Retrieval modes

- `local_first`
- `live_first`
- `local_with_live_verify`
- `archive_only`

Freshness classes

- `stable`
- `semi_volatile`
- `volatile`

默认路由指引

Stable domains

通常适合 local-first:

- courses
- course offerings
- housing
- location / service destinations
- academic calendar items

Volatile domains

不应默认把数据库归档副本当作当前真相:

- news
- announcements
- policy updates
- 多数 events
- emergency 或 temporary notices

对于这些领域，本地存储的角色应是：

- archive
- cache
- audit trail
- fallback evidence

而不是“latest/current”问题的默认第一来源。

A1 Source-First Base Model

source 层是所有 source-derived knowledge 的规范持久化形态。

1. sources

`sources` 用于定义一个 source 的长期身份和默认策略。

典型职责：

- 识别 source 身份
- 定义它属于哪个 school/domain
- 编码 authority / trust level
- 定义默认 object strategy
- 定义默认 freshness 与 retrieval 行为

典型字段：

- identity: `id`, `school_id`, `source_key`, `name`
- classification: `source_kind`, `content_domain`, `authority_level`
- default policy: `default_object_strategy`, `freshness_class`, `default_retrieval_policy`, `default_ttl_seconds`
- source location: `base_url`, `feed_url`, `api_endpoint`, `map_provider_ref`
- operations: `is_active`, `metadata`, `timestamps`

规则：`sources` 不存储权威内容正文。

2. source_snapshots

`source_snapshots` 用来捕捉某个 source 在某一时间点观察到或导入的一次版本。

典型职责：

- 记录一个版本是什么时候被捕获的
- 记录它当前是否 still current、是否 archived、expired 或 outdated
- 保存 source 侧的 currentness / verification metadata

典型字段：

- identity: `id`, `source_id`, `snapshot_key`
- capture metadata: `capture_mode`, `capture_status`, `captured_at`, `last_verified_at`

- currentness metadata: `expires_at`, `valid_from`, `valid_to`, `is_current`, `is_archived`, `is_outdated`, `archive_reason`
- source version metadata: `canonical_url`, `source_last_modified`, `etag`, `content_hash`, `raw_metadata`

规则：freshness 和 archive 语义应该落在这里，而不只是写在 tool logic 里。

3. artifacts

artifacts 是由某次 snapshot 派生出的多类型内容产物。

一次 snapshot 可以产生多个 artifact，例如：

- `raw_html`
- `clean_markdown`
- `normalized_json`
- `plain_text`
- `feed_entry`
- `object_payload`
- `map_place`
- `api_response`

典型职责：

- 以一种或多种可用形态承载 source-derived 内容
- 区分 primary / derived / evidence / object-projection 等角色
- 保存 parser / normalizer 元数据
- 标记某个 artifact 是否可搜索

典型字段：

- identity: `id`, `source_snapshot_id`
- artifact classification: `artifact_type`, `artifact_role`, `mime_type`, `language`
- content carriers: `content_text`, `content_json`, `storage_uri`
- descriptive/search fields: `title`, `summary`, `canonical_url`, `is_searchable`, `is_primary`
- normalization metadata: `normalization_status`, `parser_name`, `parser_version`, `metadata`

规则：模型必须同时支持 `content_text` 与 `content_json` 出现在同一张表中。

4. chunks

chunks 是从可搜索 artifact 派生出的仅供检索使用的切片。

典型职责：

- 支撑搜索与 citation
- 支撑 hybrid lexical + semantic retrieval
- 保留 artifact lineage 以支持证据追踪

典型字段：

- identity: `id`, `artifact_id`, `chunk_index`

- retrieval content: `chunk_text`, `search_text`, `token_count`, `heading_path`, `section_label`
- retrieval/index fields: `embedding`, `chunk_hash`, `language`, `is_active`
- provenance metadata: `metadata`, `timestamps`, 可选 `embedding_model`, `embedding_version`

规则: chunks 是 **retrieval artifacts**, 而不是 **authoritative records**。

Object-First Domain Designs

Course Domain

课程领域有且只有两个 first-class object levels:

- `course`
- `course_offering`

course

表示稳定的 catalog 事实。

典型 must-have fields:

- `id`
- `school_id`
- `subject`
- `number`
- `code`
- `title`
- `description`
- `credits`
- `canonical_url`
- `search_text`
- `source_id`
- `source_snapshot_id`
- `primary_artifact_id`

典型双层 facets:

- `department_code`
- `department_name`
- `academic_level`
- `prerequisites_text`
- `prerequisite_codes[]`
- `corequisites_text`
- `attribute_codes[]`
- `attribute_labels[]`

course_offering

表示某个学期特定开课事实。

典型 must-have fields:

- `id`
- `course_id`
- `school_id`
- `term_code`
- `status`
- `canonical_url`
- `search_text`
- `source_id`
- `source_snapshot_id`
- `primary_artifact_id`

典型双层 facets:

- `section_code`
- `class_number` 或 `crn`
- `instructor_names[]`
- `instruction_method`
- `campus`
- `meeting_days[]`
- `meeting_time_text`
- `schedule_text`
- `room_text`

明确的非目标:

- `remaining seats`
- `live enrollment`
- `waitlist counts`
- `official registration-system state`

Academic Calendar Domain

calendar 领域有且只有一个 first-class object level:

- `academic_calendar_item`

典型 must-have fields:

- `id`
- `school_id`
- `calendar_type`
- `item_type`
- `title`
- `summary`
- `start_at`
- `end_at`
- `all_day`
- `timezone`

- `status`
- `canonical_url`
- `search_text`
- `source_id`
- `source_snapshot_id`
- `primary_artifact_id`

典型 optional facets:

- `term_code`
- `academic_year`
- `applies_to_population`
- `applies_to_scope`
- `scope_labels[]`
- `related_department_codes[]`
- `related_college_codes[]`
- `related_course_codes[]`
- `description_text`
- `notes_text`
- `action_text`
- `related_urls`
- `is_deadline`
- `is_time_sensitive`

明确的非目标:

- student-specific deadline state
- registration transactions
- enrollment behavior
- workflow/rule engines

Location / Service Domain

location/service 领域有且只有一个 first-class object level:

- `location_or_service`

Identity rule:

- 每个“用户可查询的 destination”一行
- building 与 service entry 如果在用户查询和导航行为上是独立 destination, 则可以分别存在

典型 must-have fields:

- `id`
- `school_id`
- `object_type`
- `name`
- `display_name`
- `summary`

- status
- address_text
- latitude
- longitude
- canonical_url
- search_text
- source_id
- source_snapshot_id
- primary_artifact_id

典型 optional facets:

- map_provider
- map_provider_ref
- place_id
- location_hint_text
- service_type
- service_tags[]
- audience_tags[]
- hours_text
- hours_structured
- contact_text
- booking_required
- walk_in_supported
- access_notes
- campus_zone
- building_code
- parent_location_label
- related_department_codes[]
- related_service_units[]

明确的非目标:

- indoor navigation graphs
- room inventory
- queue/occupancy state
- full GIS/POI graph
- org chart modeling

Housing Domain

housing 领域有且只有一个 first-class object level:

- housing

Identity rule:

- 每个“可独立发现的官方 housing listing 或 destination”一行

- 一条记录可以表示 residence hall、apartment complex、housing community、family housing listing、graduate housing listing, 或其他官方 housing destination
- 内部变化项保留为 facets 或嵌套 option payload, 而不是再拆成更高层级对象

典型 must-have fields:

- id
- school_id
- housing_type
- name
- display_name
- summary
- status
- canonical_url
- search_text
- source_id
- source_snapshot_id
- primary_artifact_id

典型 optional facets:

- address_text
- latitude
- longitude
- campus_zone
- location_hint_text
- audience_tags[]
- eligibility_text
- gender_policy
- room_type_tags[]
- bathroom_style
- contract_type_tags[]
- meal_plan_required
- amenity_tags[]
- llc_tags[]
- price_text
- price_period
- application_url
- availability_cycle_text
- comparison_notes
- housing_policy_notes
- image_urls
- related_housing_codes[]

嵌套的解释型 option facets 可包括:

- room_options
- contract_options
- pricing_options

明确的非目标：

- realtime vacancy
 - waitlists
 - assignment workflow state
 - per-room inventory graphs
 - per-bed pricing graphs
-

Retrieval Architecture

长期方向

长期生产检索后端应为 **Supabase-native hybrid retrieval**。

这意味着：

- 用 PostgreSQL full-text search 做 lexical retrieval
- 用 pgvector 做 semantic retrieval
- 在 SQL/RPC 或 tool-layer orchestration 中进行 reciprocal rank fusion 与调优
- 用 async ingestion/indexing 取代本地 CLI-first reindexing 作为主要运维模型

当前过渡现实

repo 目前仍然同时存在两套 retrieval worlds：

- 一条较新的 Supabase-native hybrid path
- 一条较旧的 QMD/VPS path

QMD 应被视为 **transitional infrastructure**，而不是未来的 canonical backend。

Migration stance

Short term

- 保持 QMD 作为 sidecar 继续运行
- 保留兼容性与回滚安全
- 用它做 parity comparison

Medium term

- 把 ingestion 镜像写入 Supabase-native path
- 对比 retrieval quality 与 ranking behavior
- 调优 chunking、weighting、ranking 与 fallback 行为

Long term

- 当 parity 达到可接受水平后，让 QMD 退出 primary production retrieval path

Migration principle

迁移的是 **QMD 中有价值的 retrieval ideas**，而不是它的实现本体。

值得迁移的思想：

- chunking 行为与 chunk metadata discipline
- fusion / ranking 思路
- provenance-aware retrieval 行为
- context grouping 与结果整形思路

不应直接迁移的部分：

- filesystem-first ingestion 假设
- SQLite / FTS5 / sqlite-vec 实现
- 把本地 CLI indexing 当作生产环境主工作流

在 Supabase 中建议的实现形态

现有 retrieval substrate

repo 中已经存在：

- documents
- document_chunks
- hybrid_search
- hybrid_search_chunks
- keyword_search

这意味着迁移不需要从零发明 hybrid retrieval。更可能的路径是：

1. 保持当前 Supabase retrieval substrate 继续可用
2. 围绕 A1 source model 演进 ingestion
3. 把 searchable artifacts/chunks 映射进 retrieval substrate
4. 逐步提升 ranking、filtering、provenance 与 parity 行为

可能的收敛目标

随着时间推移，retrieval path 应从今天更扁平的 documents/document_chunks 模型演进到一个基于 A1 的 source lineage 形态，在该形态中：

- object-first 记录会指向 source evidence
- searchable source artifacts 可以干净地映射到 retrieval chunks
- freshness/archive 行为通过 typed fields 明确可见
- ranking 与 filtering 同时利用 retrieval signals 与 data-policy signals

Freshness and Volatile Data Rules

系统必须区别对待 volatile domains 与 stable domains。

Stable domains

通常为 local-first:

- courses
- course offerings
- housing
- location/service destinations
- academic calendar items

Volatile domains

通常为 live-first 或 local-with-live-verify:

- news
- announcements
- policy updates
- most events
- temporary notices

对数据模型的含义

数据库中保存的 volatile content 通常扮演的是:

- archive
- cache
- audit evidence
- fallback evidence

而不是 current/latest 问题的默认第一来源。

这种策略应在数据模型中显式表达, 尤其通过:

- `freshness_class`
- `default_retrieval_policy`
- `default_ttl_seconds`
- `captured_at`
- `last_verified_at`
- `expires_at`
- `is_current`
- `is_archived`
- `is_outdated`

为什么 v2 替代 db-schema-v1.md

`db-schema-v1.md` 建模的是一个更完整的校园行政世界, 在 departments、faculty、course relationships 等领域做了更强的规范化。

那个方向已经不再是当前产品的正确重心。

新系统并不是想成为:

- registrar backend
- housing assignment system
- GIS / facilities platform
- event workflow engine

它真正想成为的是一个 **tool-call-first campus assistant**，并组合：

- 少量高置信度结构化 object index
- 广覆盖的 source-first ingest 与 retrieval layer
- freshness-aware routing
- source-grounded evidence

因此，v2 选择的是面向 **assistant** 的结构，而不是面向行政完整性的结构。

推荐的后续实现顺序

1. 保持 **ADR** 对齐的命名与边界

- 不要重新把 object-first 领域做大做重

2. 先实现 **A1 source tables**

- `sources`
- `source_snapshots`
- `artifacts`
- `chunks`

3. 把当前 **retrieval bridge** 到 **A1 lineage**

- 把 searchable artifacts/chunks 映射进 Supabase retrieval
- 在需要时继续保留与 QMD 的 parity checks

4. 在 **A1 grounding** 之上实现 **object-first tables**

- `course`
- `course_offering`
- `academic_calendar_item`
- `location_or_service`
- `housing`

5. 补上 **freshness-aware routing** 字段与 **retrieval policy enforcement**

6. 把 **Supabase-native hybrid retrieval** 调优到接近 **QMD parity**，然后让 **QMD** 退出 **primary path**

相关 ADRs

- docs/adr/ADR-0004-hybrid-data-model.md
- docs/adr/ADR-0005-course-domain.md
- docs/adr/ADR-0006-source-first-base-layer.md
- docs/adr/ADR-0007-supabase-hybrid-retrieval-and-migration.md

- docs/adr/ADR-0008-academic-calendar-item.md
- docs/adr/ADR-0009-location-or-service.md
- docs/adr/ADR-0010-housing.md

已替代 / 历史背景

- docs/database/db-schema-v1.md 仍然可以作为历史背景参考，但它已经不再是校园助手的目标架构。